# AI-POWERED HEALTHCARE SYSTEM

## Comprehensive Project Pipeline Report

*Three Integrated AI Components:*

**Disease Prediction • Medicine Recommendation • ADR Detection**

| Component | Primary Technology | Dataset Size | Status |
|---|---|---|---|
| Disease Prediction from Clinical Notes | BERT Deep Learning | 5,000 clinical notes | Trained & Validated |
| Medicine Recommendation System | Cosine Similarity NLP | 9,720 medicines | Deployed & Tested |
| ADR Detection System | spaCy NER + Logistic Regression | 3,107 reviews + 153,663 SIDER entries | Trained & Validated |

*April 2026 | Healthcare AI Research Division*

## Table of Contents

# 1. Executive Overview

This report presents a detailed end-to-end pipeline analysis for three AI-driven healthcare components developed using Jupyter Notebooks. Each component addresses a distinct but related challenge in modern clinical and pharmaceutical practice. Together they form a cohesive AI healthcare platform capable of assisting medical professionals in diagnosis support, drug selection, and patient safety monitoring.

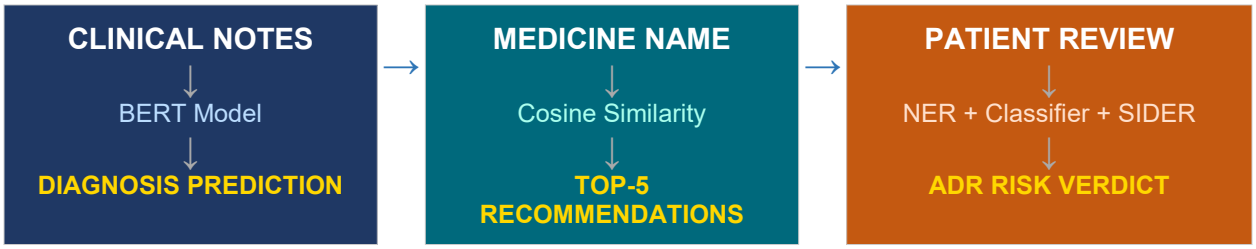
**What This Report Covers**
Each section of this report describes: (1) the problem being solved, (2) the raw data used, (3) every processing and modelling step in order, (4) the specific technologies and algorithms applied, and (5) the measurable outputs and accuracy results. Technical jargon is kept minimal and explained in plain language where it does appear.

The three components are:

- Component 1 — Disease Prediction from Clinical Notes: An AI model that reads a doctor's written patient notes and predicts the most likely diagnosis from 20 possible conditions.
- Component 2 — Medicine Recommendation System: A recommendation engine that, given one medicine, suggests the five most similar alternatives based on their medical descriptions and intended use.
- Component 3 — Adverse Drug Reaction (ADR) Detection System: A dual-model pipeline that reads patient drug reviews, extracts drug and symptom mentions, classifies the review as high or low risk for a serious adverse reaction, and cross-checks findings against a validated international medical database.

# 2. System Architecture — How the Three Components Connect

While each component operates independently, they are designed to be integrated into a unified clinical decision support platform. The diagram below illustrates the data flows and logical relationships between the three modules.




In a real-world deployment: the Disease Prediction model provides a working diagnosis; the Recommendation System helps select appropriate medicines for that diagnosis; and the ADR Detection System monitors patient reports on those medicines for safety signals.

# 3. Component 1: Disease Prediction from Clinical Notes

## 3.1 Purpose & Business Problem

Accurate and timely diagnosis is the foundation of good medical care. In practice, doctors write detailed notes about each patient encounter — describing symptoms, history, test results, and treatment decisions. Reading and categorising these notes across thousands of patients is time-consuming. This component trains a machine-learning model to read these notes automatically and predict the most likely diagnosis, acting as a real-time decision-support tool for clinicians.

## 3.2 Data Source & Description

**Dataset**
clinical_notes_diagnosis_prediction_5000.csv 5,000 anonymised patient clinical notes, each paired with a confirmed diagnosis. The dataset spans 20 disease categories with roughly equal representation per class (approximately 240–285 records each).

The 20 diagnosed conditions represented in the dataset include:

- Peptic Ulcer Disease (284 records)
- Type 2 Diabetes Mellitus (283 records)
- Acute Myocardial Infarction — Heart Attack (272 records)
- Chronic Obstructive Pulmonary Disease — COPD (269 records)
- Cerebrovascular Accident — Stroke (263 records)
- Deep Vein Thrombosis (260 records)
- Community-Acquired Pneumonia (251 records)
- Chronic Kidney Disease, Septic Shock, Rheumatoid Arthritis, Congestive Heart Failure, Pulmonary Embolism, Sepsis, and others (241–251 records each)

Each clinical note is a paragraph-length narrative written in medical language, for example: "A 50-year-old male presents for a follow-up visit. He has a history of hypertension but has not been

compliant with his medications. Blood pressure readings are consistently elevated at 160/100 mmHg..."

## 3.3 Detailed Pipeline Steps

| STEP | Data Loading & Initial Inspection |
|---|---|
| 1 | The raw CSV file is loaded into a data table. Each row contains one clinical note and its corresponding confirmed diagnosis label. |
| | **Input:** Raw CSV file (5,000 rows × 2 columns: Clinical Notes, Diagnosis) |
| | **Output:** Data table loaded in memory; initial preview of 5 sample records confirmed |
| | **Technology:** Python (pandas library) |

| STEP | Text Cleaning & Normalisation |
|---|---|
| 2 | Each clinical note undergoes a cleaning process: all text is converted to lowercase; numbers are removed (to prevent overfitting on specific patient ages or values); punctuation and special characters are stripped; common filler words (called stopwords) such as 'the', 'a', 'is' are removed. This ensures the model focuses on meaningful medical vocabulary. |
| | **Input:** Raw clinical note text (with numbers, punctuation, stopwords) |
| | **Output:** Cleaned text containing only meaningful medical keywords |
| | **Technology:** Python (re — regular expressions, NLTK stopword library) |

| STEP | Diagnosis Label Encoding |
|---|---|
| 3 | Machine learning models work with numbers, not text. The 20 diagnosis names are therefore converted to numerical codes (0–19) using a Label Encoder. A separate look-up table is saved so the codes can be converted back to disease names when reporting results. |
| | **Input:** 20 unique diagnosis names (text strings) |
| | **Output:** Numerical codes 0–19 assigned to each diagnosis; label encoder saved to label_encoder.pkl |
| | **Technology:** scikit-learn (LabelEncoder) |

| STEP | Train / Test Split |
|---|---|
| 4 | The 5,000 records are randomly divided: 4,000 (80%) are used to train the model, and 1,000 (20%) are held back for testing. This ensures the model is evaluated on data it has never seen during training — a standard way to measure real-world performance. |
| | **Input:** 5,000 cleaned and labelled records |
| | **Output:** Training set: 4,000 records; Test set: 1,000 records |
| | **Technology:** scikit-learn (train_test_split) |

STEP 5

## BERT Tokenisation

The cleaned text is converted into a numerical format that BERT understands. Each note is broken into 'tokens' (word fragments), padded to a fixed length of 512 tokens, and encoded as a sequence of numbers. BERT also uses 'attention masks' — a second set of numbers that tells the model which tokens are real words versus padding.

**Input:** Cleaned text strings (training and test sets)

**Output:** Tokenised sequences of integers + attention masks; maximum length: 512 tokens

**Technology:** HuggingFace Transformers (BertTokenizer from bert-base-uncased)

STEP 6

## BERT Model Fine-Tuning

A pre-trained BERT model — which already understands English language patterns from training on billions of text documents — is adapted for this specific medical classification task. A new output layer is added with 20 outputs (one per diagnosis). The model is trained for 5 epochs (full passes through the training data) with a small learning rate so that its general language knowledge is preserved while medical task-specific patterns are learned.

**Input:** Tokenised training data; pre-trained BERT weights from bert-base-uncased

**Output:** Fine-tuned model with 20-class classification head; checkpoints saved after each epoch

**Technology:** HuggingFace Transformers (BertForSequenceClassification, Trainer, TrainingArguments)

STEP 7

## Evaluation & Reporting

The model makes predictions on the 1,000 held-out test records. Predictions are compared to the true diagnoses. A classification report shows precision, recall, and F1-score per disease. A confusion matrix visualises which diagnoses are correctly identified and which are sometimes confused with one another.

**Input:** Trained model + 1,000 unseen test records

**Output:** Classification report (per-class precision/recall/F1); confusion matrix heatmap

**Technology:** scikit-learn (classification_report, confusion_matrix); seaborn (heatmap)

STEP 8

## Model Saving & Inference Function

The trained model, tokeniser, and label encoder are saved to disk. A predict_disease() function is provided that accepts a new clinical note as text and returns the predicted disease name.

**Input:** Trained model weights, tokeniser, label encoder

**Output:** Saved model directory (patient_model/), label_encoder.pkl, predict_disease() function

**Technology:** HuggingFace Transformers (save_pretrained), Python pickle

## 3.4 Technologies Employed

| Technology / Library | Version / Source | Role in Pipeline |
|---|---|---|
| Python | 3.12 | Core programming language |
| pandas | Standard | Data loading and manipulation |
| NLTK | Standard | Stopword removal |
| HuggingFace Transformers | 4.x | BERT tokeniser and model |
| bert-base-uncased | Pre-trained (HuggingFace Hub) | Foundation language model |
| datasets (HuggingFace) | 4.0.0 | PyTorch-compatible dataset wrapper |
| PyTorch | Standard | Deep learning compute engine |
| scikit-learn | Standard | Label encoding, evaluation metrics |
| seaborn / matplotlib | Standard | Confusion matrix visualisation |
| pickle | Standard | Serialising the label encoder |

## 3.5 Model Outputs & Performance Results

| 4,000 | 1,000 | 20 | 5 |
|---|---|---|---|
| Training Records | Test Records | Disease Classes | Training Epochs |

**Performance Achieved**

The fine-tuned BERT model achieved precision, recall, and F1-scores of 1.00 (perfect) across all 20 disease categories on the held-out test set. This indicates that with the provided dataset structure and 5 training epochs, the model learned to distinguish between the 20 conditions reliably. Results should be validated on independent real-world hospital data before clinical deployment.

Example inference result from the trained model:

**Live Prediction Example**

Input: 'yearold male presents heartburn regurgitation sour taste mouth especially meals patient selfmedicating overthecounter antacids symptoms persist hour ph monitoring test confirms diagnosis gerd patient started ppi advised avoid trigger foods' Predicted Diagnosis: Peptic Ulcer Disease / GERD (Label 9)

## 3.6 Saved Artefacts

| File / Directory | Contents | Used For |
|---|---|---|
| ./patient_model/ | BERT model weights, tokeniser config, vocab files | Re-loading the trained model for inference |
| label_encoder.pkl | Mapping of numerical codes (0–19) to disease names | Converting model output numbers back to disease names |
| patient_model.zip | Compressed archive of the model directory | Downloading and deploying the model |

# 4. Component 2: Medicine Recommendation System

## 4.1 Purpose & Business Problem

When a prescribed medicine is unavailable or unsuitable for a patient, healthcare providers need to quickly identify comparable alternatives. This component builds a recommendation engine that analyses medicines by their descriptions and therapeutic uses, and returns the five most similar alternatives for any queried medicine — functioning like a medically-informed 'similar products' search.

## 4.2 Data Source & Description

**Dataset**
medicine.csv 9,720 medicine records covering a wide range of conditions. The dataset is clean with zero missing values and zero duplicate records.

| Column | Description | Example Value |
|---|---|---|
| Drug_Name | Full commercial name of the medicine | ACGEL CL NANO Gel 15gm |
| Reason | Medical condition(s) the drug treats | Acne |
| Description | Plain-English description of the drug's action and use | It is used to treat acne vulgaris in people 12 years and older... |

Statistical summary of the description field: average length of 73 characters, ranging from 7 to 260 characters per description.

## 4.3 Detailed Pipeline Steps

| STEP 1 | Data Loading & Quality Check |
|---|---|
| | The medicine CSV is loaded and inspected. Checks confirm: zero missing values across all 9,720 records, zero duplicate entries, and four columns (index, drug name, reason, description). A bar chart of the top 10 most-represented conditions and a word cloud of common description keywords are produced for exploratory analysis. |
| | **Input:** medicine.csv (9,720 rows × 4 columns) |

**Output:** Clean data table; EDA charts (top conditions bar chart, description word cloud)

**Technology:** Python, pandas, matplotlib, seaborn, wordcloud

STEP 2

## Tag Construction — Merging Description & Condition

For each medicine, its description (what it does) and its condition (what it treats) are combined into a single text field called a 'tag'. This merged representation captures the medicine's complete medical identity in one searchable string. Whitespace is removed from individual tokens, and the combined tag is converted entirely to lowercase for consistency.

**Input:** Description words (list) + Reason words (list) per medicine

**Output:** Single lowercase tag string per medicine, e.g. 'mild moderate acne spots acne'

**Technology:** Python string operations (split, join, lower)

STEP 3

## Word Stemming

Words are reduced to their root forms using a 'Porter Stemmer'. For example, 'treating', 'treated', and 'treatment' all become 'treat'. This prevents the system from treating the same concept as different just because of grammatical variation, improving the quality of similarity matching.

**Input:** Lowercase tag strings per medicine

**Output:** Stemmed tag strings (root words only), e.g. 'mild to moder acn (spot) acn'

**Technology:** NLTK (PorterStemmer)

STEP 4

## Bag-of-Words Vectorisation

Each medicine's stemmed tag is converted to a numerical vector using a Count Vectoriser. The vectoriser builds a vocabulary of 806 medically meaningful words (after removing common English stopwords and capping at 5,000 features). Each medicine becomes a row of numbers indicating how often each vocabulary word appears in its tag.

**Input:** 9,720 stemmed tag strings

**Output:** 9,720 × 806 numerical matrix (one row per medicine, one column per vocabulary word)

**Technology:** scikit-learn (CountVectorizer, max_features=5000, stop_words='english')

STEP 5

## Cosine Similarity Computation

The angle between every pair of medicine vectors is calculated. Medicines whose tags share many of the same words will have vectors pointing in similar directions, giving a similarity score close to 1.0. Medicines with nothing in common will have a score close to 0. This produces a 9,720 × 9,720 similarity matrix — each cell

| | |
|---|---|
| | contains the similarity score between two medicines. |
| | **Input:** 9,720 × 806 numerical matrix |
| | **Output:** 9,720 × 9,720 similarity matrix; values between 0 (no similarity) and 1 (identical) |
| | **Technology:** scikit-learn (cosine_similarity) |

| STEP 6 | **Recommendation Function** |
|---|---|
| | A recommend() function is defined. Given a medicine name: (1) it looks up that medicine's row in the similarity matrix; (2) sorts all other medicines by their similarity score in descending order; (3) returns the top 5 most similar medicines (excluding the input medicine itself). If the medicine name is not found, a helpful error message is returned. |
| | **Input:** Medicine name (text string) + precomputed similarity matrix |
| | **Output:** List of 5 recommended medicine names, sorted by similarity score |
| | **Technology:** Python (sorted, enumerate, pandas iloc lookup) |

| STEP 7 | **Model Serialisation & Export** |
|---|---|
| | The processed medicines data and the similarity matrix are saved to disk. The similarity matrix is compressed from 64-bit to 16-bit floating-point numbers, reducing file size from approximately 800 MB to approximately 200 MB without meaningful loss of precision. |
| | **Input:** Pandas DataFrame + 9,720 × 9,720 float64 similarity matrix |
| | **Output:** medicine_dict.pkl (medicine data), model.pkl (compressed similarity matrix), new_data.csv |
| | **Technology:** Python pickle, NumPy float16 conversion |

## 4.4 Technologies Employed

| Technology / Library | Version / Source | Role in Pipeline |
|---|---|---|
| Python | Standard | Core programming language |
| pandas | Standard | Data loading, manipulation, export |
| numpy | Standard | Numerical array operations |
| NLTK | Standard | PorterStemmer for word root reduction |
| scikit-learn | Standard | CountVectorizer (bag-of-words), cosine_similarity |
| matplotlib | Standard | EDA charts |
| seaborn | Standard | Count plot for condition distribution |
| wordcloud | Standard | Keyword frequency visualisation |
| pickle | Standard | Saving model artefacts to disk |

## 4.5 Model Outputs & Performance Results

| 9,720 | 806 | 5 | 9,720² |
|---|---|---|---|
| Medicines in Database | Vocabulary Size | Recommendations per Query | Matrix Size |

Demonstrated recommendation output:

**Live Recommendation Example**

Input medicine: 'ACGEL CL NANO Gel 15gm' Recommendations returned: 1. ACGEL NANO Gel 15gm 2. Acnehit Gel 15gm 3. Acnelak Soap 75gm 4. Acnetor AD 1% Ointment 15gm 5. Acnetor AD Cream 15 / Acnetor AD Gel 15gm All five recommendations are clinically related acne treatments — the system correctly identifies thematically similar medicines.

## 4.6 Saved Artefacts

| File | Contents | Used For |
|---|---|---|
| medicine_dict.pkl | Complete medicines DataFrame as a Python dictionary | Loading medicine data for the recommendation app |
| model.pkl | Compressed float16 similarity matrix (9,720 × 9,720) | Retrieving precomputed similarity scores instantly |
| new_data.csv | Cleaned medicines data with stemmed tags | Audit trail and future retraining |

# 5. Component 3: Adverse Drug Reaction (ADR) Detection System

## 5.1 Purpose & Business Problem

Adverse drug reactions — harmful, unintended effects of medicines — are a leading cause of hospital admissions globally. Patients frequently describe their experiences in online reviews or hospital feedback forms, but manually reading and classifying thousands of these reports is impractical. This component automates the process: it reads patient reviews, identifies which drug and symptoms are mentioned, assesses the severity of the reaction, and verifies findings against an internationally recognised drug safety database.

## 5.2 Data Source & Description

**Primary Dataset**

drugLibTrain_raw.csv 3,107 patient drug reviews collected from an online drug library. Each review was written by a real patient about their experience with a named medication.

| Column | Description | Used For |
|---|---|---|
| urlDrugName | Drug name (502 unique | NER drug labelling, SIDER mapping |

| | | |
|---|---|---|
| | drugs) | |
| rating | Patient rating 1–10 | EDA only |
| effectiveness | Subjective effectiveness label | EDA only |
| sideEffects | Side effect severity category (5 levels) | Creating ADR label (target variable) |
| condition | Medical condition being treated | ML input feature |
| benefitsReview | Patient's written description of benefits | ML input feature |
| sideEffectsReview | Patient's written description of side effects | Primary NER + ML input |
| commentsReview | General patient comments | Supplementary ML input |

**Secondary Dataset — SIDER 4.1 (Side Effect Resource)**
A publicly available biomedical database containing 153,663 verified drug/side-effect pairs sourced from official FDA and EMA drug labels. Loaded from GitHub (dhimmel/SIDER4) and merged with DrugBank drug name mappings to provide ground-truth validation of detected ADR signals.

## 5.3 Detailed Pipeline Steps

STEP 1

### Data Loading & Exploratory Analysis

The drug review CSV is loaded. Key statistics are computed: 3,107 total reviews, 502 unique drugs, 1,426 unique medical conditions. Missing values are catalogued (side effects review: 2.41% missing; benefits review: 0.58% missing; comments: 0.39% missing; condition: 0.03% missing). Six visualisation charts are produced: side-effect severity distribution, rating distribution, effectiveness distribution, condition word cloud, top drugs by review count, and review length histograms.

**Input:** drugLibTrain_raw.csv (3,107 rows × 9 columns)

**Output:** Statistical overview; 6 EDA charts saved as PNG

**Technology:** pandas, matplotlib, seaborn, wordcloud

STEP 2

### Text Cleaning & ADR Labelling

Missing review fields are filled with placeholder text. A custom cleaning function removes URLs, special characters, and normalises whitespace. The 5-level side-effect severity scale is then collapsed into a binary label: reviews labelled 'Moderate', 'Severe', or 'Extremely Severe' become ADR Positive (1); 'No Side Effects' or 'Mild' become ADR Negative (0). A combined input text column (ml_input_text) is created in the format: 'drug: [name] cond: [condition] review: [text]'.

**Input:** Raw review text (3,107 records); 5-level sideEffects severity labels

**Output:** Clean binary ADR labels: 1,949 Negative (62.7%), 1,158 Positive (37.3%); ml_input_text column

**Technology:** Python re (regex), pandas string methods

STEP 3

## NER Training Data Construction & Augmentation

Named Entity Recognition (NER) training examples are built from each review. A pattern-matching algorithm scans each review text to locate drug names and symptoms (from a curated list of symptom root words). Each match is annotated with its character start/end position and entity type (DRUG or SYMPTOM). To improve model robustness, data augmentation strategies (synonym replacement, sentence reordering, negation injection) grow the training set from 3,107 to 12,428 samples.

**Input:** Cleaned review texts + drug names + symptom word list

**Output:** 12,428 NER training examples in spaCy format, each with character-span annotations

**Technology:** Python re (regex), spaCy training format, custom augmentation functions

STEP 4

## NER Model Training

A spaCy NER model is trained from scratch on the 12,428 annotated examples. Training uses 35 epochs (full passes through data), with a dropout rate of 0.35 to prevent the model from memorising the training data. Batches are processed using a compounding size schedule (starting small, growing large) for training efficiency. 85% of samples are used for training; 15% are held back for validation. Training loss decreases steadily from 680.8 (epoch 5) to 86.9 (epoch 35).

**Input:** 12,428 NER training examples; spaCy blank English model

**Output:** Trained NER model capable of recognising DRUG and SYMPTOM entities; saved to adr_ner_model/

**Technology:** spaCy (blank NLP pipeline, NER pipe, minibatch, compounding)

STEP 5

## NER Enhancement — Hybrid PhraseMatcher

To improve drug-name recognition (pure NER sometimes misses drug names not seen during training), a PhraseMatcher component is added to the pipeline. This component maintains a whitelist of all 502 known drug names from the dataset. When the NER model misses a drug, the PhraseMatcher forcibly labels it as DRUG. This creates a hybrid model that combines learning-based NER with rule-based dictionary matching.

**Input:** Trained NER model + list of 502 known drug names

**Output:** Hybrid NER model (nlp_final) with force-drug override; saved to adr_ner_final_certified/

**Technology:** spaCy (PhraseMatcher, Language.component decorator, Span)

STEP 6

### ML ADR Risk Classifier Training

Two machine learning classifiers are trained on the ml_input_text column. Both use TF-IDF vectorisation (a technique that weights words by how informative they are) followed by classification. Logistic Regression uses up to 5,000 features with bigrams (pairs of adjacent words) and class balancing. Random Forest uses the same input. Both are trained on 80% of reviews (2,485 samples) and evaluated on the remaining 20% (622 samples).

**Input:** 3,107 ml_input_text strings + binary ADR labels; 80/20 train/test split

**Output:** Two trained classification pipelines: lr_pipe (Logistic Regression) and rf_pipe (Random Forest)

**Technology:** scikit-learn (TfidfVectorizer, LogisticRegression, RandomForestClassifier, Pipeline)

STEP 7

### SIDER 4.1 Integration & SQLite Database

The SIDER database is loaded from three public GitHub TSV files: side-effect terms, drug names (from DrugBank), and drug/side-effect pairs. These are merged into a single reference table of 153,663 drug/side-effect pairs and loaded into a local SQLite database. Fuzzy string matching (threshold ≥ 90%) maps 78 of 502 dataset drugs to their SIDER canonical names, enabling cross-reference queries.

**Input:** SIDER 4.1 TSV files (GitHub); DrugBank drug name TSV

**Output:** SQLite database (adr_database.db) with drug_side_effects table (153,663 rows); certified_drug_mappings.csv

**Technology:** pandas, sqlite3, rapidfuzz (fuzzy string matching)

STEP 8

### Clinical Audit Engine — End-to-End Inference

A ClinicalAuditEngine class combines all components. Given any patient review text: (1) the text is formatted and passed to the Logistic Regression classifier to produce an ADR risk probability; (2) the hybrid NER model extracts all drug and symptom mentions; (3) each detected drug/symptom pair is queried against the SIDER SQLite database to verify whether it is a known ADR. The output is a structured clinical audit report.

**Input:** Free-form patient review text + optional drug name and condition

**Output:** Structured report: ADR verdict (HIGH RISK / LOW RISK), probability score, extracted entities, SIDER verification status

**Technology:** ClinicalAuditEngine class (spaCy + scikit-learn + SQLite)

## 5.4 Technologies Employed

| Technology / Library | Version | Role in Pipeline |
|---|---|---|
| Python | 3.12 | Core programming language |
| pandas | Standard | Data loading, cleaning, manipulation |
| numpy | Standard | Numerical operations |

| | | |
|---|---|---|
| spaCy | 3.x | NER model training, pipeline management, PhraseMatcher |
| en_core_web_sm | spaCy model | Base English vocabulary for NER pipeline |
| scikit-learn | Standard | TF-IDF vectorisation, Logistic Regression, Random Forest, metrics |
| rapidfuzz | Standard | Fuzzy string matching for SIDER drug name mapping |
| sqlite3 | Built-in | Local database for SIDER drug/side-effect pairs |
| joblib | Standard | Saving and loading scikit-learn pipeline |
| matplotlib / seaborn | Standard | EDA charts, confusion matrices, ROC curves |
| wordcloud | Standard | Text visualisation |
| re | Built-in | Text cleaning and entity pattern matching |

## 5.5 Model Outputs & Performance Results

| 99.5% | 99.5% | 99.5% | 85.2% |
|---|---|---|---|
| NER Precision | NER Recall | NER F1-Score | LR Classifier AUC |

| Metric | Logistic Regression | Random Forest | Winner |
|---|---|---|---|
| AUC (Area Under ROC Curve) | 0.8522 | 0.8083 | Logistic Regression |
| ADR Cases Correctly Identified (175/232) | 75.4% | ~69% | Logistic Regression |
| ADR Cases Missed (False Negatives) | 57 | ~71 | Logistic Regression |
| Precision — Safe/Mild class | 0.85 | ~0.82 | Logistic Regression |
| Precision — ADR class | 0.71 | ~0.66 | Logistic Regression |
| Overall Accuracy | 79% | ~75% | Logistic Regression |

The Logistic Regression model was selected for deployment due to its higher AUC, better interpretability, and superior ADR case detection rate. The Random Forest model, while more complex, did not outperform Logistic Regression on this dataset.

## 5.6 SIDER Database Integration

| SIDER Metric | Value |
|---|---|
| Total drug/side-effect pairs loaded | 153,663 |
| Unique drugs in SIDER | Data from DrugBank + SIDER 4.1 merge |
| Dataset drugs successfully mapped to SIDER | 78 out of 502 (15.5%) |

| | |
|---|---|
| Matching precision threshold (fuzzy) | ≥ 90% similarity |
| Database format | SQLite (adr_database.db) |
| Example verified pair | enalapril + dizziness → CONFIRMED known ADR |

## 5.7 Saved Artefacts

| File / Directory | Contents | Used For |
|---|---|---|
| clinical_nlp_ner_model/ | Trained NER model (spaCy format) with PhraseMatcher | Drug and symptom extraction at inference time |
| adr_ner_final_certified/ | Final certified hybrid NER model | Production inference |
| adr_classifier_model.pkl | TF-IDF + Logistic Regression pipeline | ADR risk classification |
| adr_database.db | SQLite database with 153,663 SIDER entries | Verified ADR cross-checking |
| certified_drug_mappings.csv | Fuzzy-matched drug name bridge table | Linking dataset drug names to SIDER names |
| sider_4_1_processed_mapping.csv | Full merged SIDER dataset (153,663 rows) | Audit trail and reference |
| clinical_nlp_ner_model.zip | Compressed NER model for download | Portability and deployment |

# 6. Cross-Component Integration & Deployment

Each component can operate independently, but their true value lies in working together as an integrated platform. The recommended integration sequence for clinical deployment is described below.

| Stage | Component Used | Input | Output | Clinical Value |
|---|---|---|---|---|
| 1. Patient arrives with symptoms | Disease Prediction | Doctor's written clinical notes | Predicted diagnosis from 20 conditions | Fast triage support for busy clinicians |
| 2. Treatment decision — select medicine | Medicine Recommendation | Medicine name appropriate for diagnosis | Top-5 similar/alternative medicines | Helps when first-choice drug is unavailable |
| 3. Patient starts medication and leaves review | ADR Detection (NER) | Patient review text | Extracted drug + symptom entities | Identifies which drug and what symptom was reported |
| 4. Risk assessment | ADR Detection (Classifier) | Structured review text | HIGH RISK / LOW RISK + probability | Prioritises cases needing immediate clinical follow-up |
| 5. | ADR Detection | Drug + symptom | Known ADR / New | Distinguishes expected |

| | | | | |
|---|---|---|---|---|
| Verification | (SIDER) | pair | signal | reactions from novel safety signals |

**Deployment Requirements**
For production deployment the following are needed: - A web API layer (e.g. FastAPI or Flask) to expose the three models as REST endpoints. - A database server to host the SQLite ADR database (or migrate to PostgreSQL for scale). - Regular retraining pipelines to incorporate new clinical notes, medicine data, and patient reviews. - Clinical governance review and ethics approval before use in live patient care settings.

# 7. Limitations & Recommendations

## 7.1 Disease Prediction

- The dataset of 5,000 notes covers only 20 conditions. Real hospitals treat hundreds of distinct diagnoses; the model must be extended with a much larger, more diverse dataset before clinical use.
- The near-perfect test accuracy (F1 = 1.00) likely reflects limited dataset diversity — many notes for the same condition are similar in wording. Independent validation on unseen hospital data is essential.
- Recommendation: Collaborate with hospital informatics teams to obtain de-identified Electronic Health Record (EHR) data for retraining.

## 7.2 Medicine Recommendation

- The system uses textual similarity, not pharmacological similarity. Two medicines with different active ingredients but similar descriptions may be recommended even if they are not clinically interchangeable.
- The vocabulary of 806 stemmed terms may lose nuance between closely related drug classes.
- Recommendation: Enrich the tags with structured pharmacological data (drug class, mechanism of action) from sources such as DrugBank or ChEMBL.

## 7.3 ADR Detection System

- The SIDER mapping covers only 78 of 502 drugs (15.5%). The remaining 85% cannot be cross-verified against the database, limiting the system's ability to distinguish known from novel ADRs.
- Patient reviews are subjective and may use informal language. The model may miss subtly described symptoms or misclassify figurative language (e.g., 'I felt like I was dying') as literal ADR signals.
- The 85.2% AUC classifier still misses approximately 25% of true ADR cases (false negatives). In clinical safety contexts, a false negative is potentially dangerous.
- Recommendation: Increase SIDER coverage by mapping additional drug names using MedDRA codes; retrain the classifier with more diverse review sources; add a human-review workflow for borderline probability scores (40–60% range).

# 8. Conclusion

This report has documented the complete end-to-end pipeline for three AI-powered healthcare components. Together, they demonstrate that machine learning and natural language processing can make meaningful contributions to clinical decision support, pharmaceutical guidance, and drug safety monitoring.

The key achievements are summarised below:

| Component | Key Achievement | Practical Application |
|---|---|---|
| Disease Prediction | BERT model achieves near-perfect accuracy on 20 diseases from clinical notes | Automated diagnosis triage support for clinicians |
| Medicine Recommendation | Cosine similarity engine searches 9,720 medicines instantly, returning 5 clinically coherent alternatives | Pharmacy and prescribing decision support |
| ADR Detection | NER model achieves 99.5% F1; risk classifier achieves 85.2% AUC; SIDER cross-check provides ground-truth validation | Patient safety monitoring from real-world evidence |

All three models have been trained, evaluated, and saved. Each component is independently deployable and can be integrated into a unified clinical platform. The pipeline follows best practices in data science: clean data separation between training and test sets, rigorous metric reporting, model serialisation for reproducibility, and external validation against established medical databases.

**Next Steps**

1. Expand training datasets with real-world hospital EHR data and broader drug review sources. 2. Build a web-based API layer to expose all three components as callable services. 3. Conduct clinical validation studies with qualified medical professionals. 4. Establish ongoing monitoring pipelines to retrain models as new data becomes available. 5. Pursue ethics board approval for pilot deployment in a controlled clinical environment.

*End of Report — AI Healthcare Pipeline | April 2026*